Egyptian Informatics Journal 30 (2025) 100678

Contents lists available at ScienceDirect

Egyptian Informatics Journal

journal homepage: www.sciencedirect.com

Full Length Article

The application and performance optimization of multi-controller-based load balancing algorithm in computer networks

Fengfeng Guo [a,*], Ailing Ye [b,c]

[a] Computer Information Department of Suzhou Vocational and Technical College, Suzhou 234000, China
[b] Network Engineering College & Wuhu Institute of Technology, Wuhu 241003, China
[c] Faculty of Information Science & Technology Universiti Kebangsaan Malaysia, Bangi 43600, Malaysia

ARTICLE INFO

Keywords:
Load balancing
Multiple controllers
Genetic-bird swarm optimization
Computer networks

ABSTRACT

This paper addresses the critical issue of network congestion caused by the increase in network traffic in contemporary society. The computer networks serve as the foundation for information exchange and online services, and their efficiency is essential. Traditional load-balancing algorithms face challenges in handling dynamic workloads, leading to inefficient resource utilization and extended response time. To address this problem, a novel method called Genetic-Bird Swarm Optimization (GBSO) is introduced, focusing on multi-controller-based load balancing. This method involves problem modeling, analysis, and selection processes, including the selection of switches and target controllers within the network segment. The results showed that the throughput of the proposed GBSO method was about 3800, and the load index after load balancing was 0.6, indicating that the workload distribution was balanced. The accuracy of the proposed GBSO algorithm was 92.15 %, the precision was 89 %, the recall rate was 88 %, and the F1 score was 85 %, all of which were higher than the existing Naive Bayes algorithm. This study emphasizes the importance of load balancing in optimizing computer network performance. The new algorithm proposed in this article provides a reliable solution for uniform network traffic distribution, reducing the limitations of existing methods.

1. Introduction

The proliferation of devices in modern computer networks and the exponential increase in data traffic have increased the need for effective load-balancing solutions. Although the traditional load balancing algorithm based on single controller is successful, it may need to be adjusted to adapt to the scale and complexity of contemporary network systems. With the application of Software-Defined Networking (SDN), new paradigms have emerged that multiple controller designs can be installed to improve load balancing performance [1]. Computer networks are the foundation of modern information systems in today's ever-changing digital world, allowing seamless data sharing, resource utilization, and communication across various devices and platforms [2]. With the increasing demand for effective and reliable network services, the importance of the best use of network resources is also increasing. Load balancing is essential for efficiently allocating network traffic and workloads among various network components [3]. This guarantees excellent user experiences, better speed, and less network congestion. In this respect, a viable approach to tackling the issues related to contemporary network infrastructures is the application and performance optimization of multi-controller-based load-balancing algorithms [4].

As network designs have become increasingly sophisticated and network traffic has become more dynamic, load-balancing solutions have advanced dramatically. Modern load-balancing algorithms using many controllers to properly manage and distribute network traffic are known as multi-controller-based load-balancing algorithms [5]. Compared with conventional single-controller models, multi-controller systems provide improved scalability, fault tolerance, and adaptability, making them especially well-suited for today's complex and demanding network scenarios [6]. This study aims to explore, analyze, and improve controller-based load-balancing methods in computer networks. These algorithms utilize the functions of multiple controllers to allocate resources more effectively, reduce latency and improve the overall network performance [7]. By analyzing the principles, advantages and potential disadvantages of these algorithms, this paper tries to delve into their fundamental ideas.

Allocating incoming traffic to the best network components wisely is

* Corresponding author.
E-mail address: 13855725226@163.com (F. Guo).

https://doi.org/10.1016/j.eij.2025.100678
Received 6 December 2024; Received in revised form 24 March 2025; Accepted 6 April 2025
Available online 14 April 2025
1110-8665/© 2025 THE AUTHORS. Published by Elsevier BV on behalf of Faculty of Computers and Artificial Intelligence, Cairo University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

one of the fundamental issues in load balancing. Each controller in multi-controller-based algorithms evaluates the network circumstances, workload distribution, and resource availability, and introduces a dynamic and distributed decision-making process [8]. Real-time modifications are made possible by collaborative decision-making among controllers, ensuring that traffic is routed appropriately to avoid overload of specific network components, and utilize underutilized resources [9]. This study examines the decision-making process of these algorithms to reveal the subtleties of their operation and evaluate their effectiveness in different network situations [10]. Furthermore, a crucial component of this study is performance optimization. The importance of optimizing load-balancing algorithms for different conditions and workloads increases as network environments become more complex and diverse.

Although some research achievements have been made in the field of computer network load balancing, the real-time and accuracy of the decision-making process of existing multi-controller load balancing algorithms still need to be improved when facing large-scale complex network environments. The research on the adaptability of load balancing algorithms in different network scenarios is not deep enough. Based on the research gap mentioned above, to alleviate network congestion brought by rising traffic, this research introduces a unique strategy known as Genetic-Bird Swarm Optimization (GBSO). The study seeks to improve resource consumption and network efficiency by concentrating on multi-controller-based load balancing, providing a solid answer for better performance.

1.1. Contribution to this paper

- This study proposes a unique method to achieve payload balancing in computer networks, called GBSO, which solves the traffic increase caused by network congestion.
- Dynamic workloads are complex for traditional methods, resulting in delayed answers and inefficient resource usage. To overcome this, GBSO uses a multi-controller solution, improving resource efficiency and speeding up response time.
- In GBSO, switch and target controllers are assigned through a selection procedure, problem modeling, and analysis. This distinct method optimizes the load distribution across network segments, resulting in better performance.
- The effectiveness of the research is confirmed through evaluation measures, including accuracy, precision, recall, F1 score, load balance rate, and network latency. The role of GBSO in enhancing computer network efficiency is demonstrated.

The additional divisions of the following are this article: Part 2 introduces related works, Part 3 discusses the methodology, Part 4 assesses the efficiency of the proposed method, part 5 discusses the paper, and Part 6 concludes the article.

2. Related works

The paper [11] offered a Deep Reinforcement Learning (DRL) method, which used Bio-Inspired Deep Belief Architecture (BDBA) and improved routes in SDN by introducing the concept of human emotional learning. The suggested method optimized each Bio-Inspired RBM using a reward function that captures network policies, allowing for real-time route optimization without prior information. Compared with traditional approaches, the proposed DRL-based Bio-Inspired method efficiently optimized routes in dispersed SDNs. The complexity and variety of network infrastructures may impact the performance of the suggested system, necessitating further development to handle some extreme scenarios.

The research [12] improved the security and efficiency of wireless 5G networks using a modified blockchain, handover authentication strategy, and other methods, including CapsNet, NFV-enabled switches, and SAC algorithm for packet validation and load balancing. The suggested model used a modified blockchain for user authentication, the QUARK algorithm for creating hashes, edge servers for handover, OpenFlow switches for validating flow rules, CapsNet for packet classification, NFV-enabled vSwitches for load management, and the SAC and HBO algorithms for load balancing and suspicious packet validation, respectively. Compared with previous methods, the created model, implemented in NS-3, performed better, delivering improved quality of service metrics and security metrics.

The research [13] proposed a simple controller placement algorithm, and used the cost function based on topology and traffic distribution to evaluate the efficiency of the algorithm, so as to solve the load balancing problem in the Software Defined Wide Area Network (SD-WAN) control plane. The proposed Simulated Annealing Partition-based K-Means (SAPKM) approach reduced average load and improved load balancing indices by strategically allocating controllers while considering topology and flow distribution. Experimental simulations on various network topologies showed that the SAPKM simultaneously increased propagation latency and network resilience while improving load balancing, lowering average load, and reducing average burden. The limitations of this study may include special assumptions about the characteristics of the network and the scalability of the proposed algorithm in a broader and complex network.

The research [14] established load balancing among controller clusters in SDNs using a non-cooperative game theory-based mechanism, maximizing utility functions related to load balance, average Latency, and switch migration cost. A similarity operator was added to the improved Multi-Objective Optimization Genetic Algorithm (GAIMO) to improve the convergence and accuracy of the algorithm and realize the dynamic migration of SDN switches. The technique optimized controller resource use, lowered communication overhead, switched migration costs, and enhanced overall network performance. The drawbacks of this method included sensitivity initial parameters and potential complexity in large-scale networks.

The study [15] suggested a latency-based load balancing method to handle the load imbalance among numerous SDN controllers. The algorithm considered controller loads and real-time latency to simultaneously resolve multiple overloads and achieved load balancing on the SDN control plane. Compared with existing algorithms, the latency was reduced by 25 %, improving network performance. The summary did not specify the algorithm implementation or any potential difficulties.

The paper [16] improved multi-domain projectile flight test networks by solving asymmetric load problems among multi-controllers using more effective Ant Colony Algorithm (IACO) that blends ant colony and artificial fish swarm methods. The suggested IACO transformed a controller-switch translation problem to a traveling salesperson problem, which differed from a traditional ant colony and shared decision algorithms, ensuring load balancing and improving cluster reliability. IACO outperformed other algorithms, as simulation showed improved multi-controller cluster throughput, faster controller request event response time, and better load balancing. The IACO technique may still need to be enhanced when handling cases with highly complex or dynamically changing networks.

The study [17] optimized switch-to-controller assignments for effective load balancing and decreased round trip time to solve controller performance degradation in SD-WAN. The Distributed Hungarian-based Assignment Protocol (DHAP) for multi-controller settings employed the strategy. Regarding load balancing and flow setup time, the suggested DHAP method outperformed parallel schemes, increasing controller performance and network effectiveness. The method may ignore other possible network optimization factors because it primarily focuses on load balancing and switch-to-controller assignment optimization.

The research [18] provided an Optimized load-balancing-based Admission Control Mechanism (Opt-ACM) for SDN that used for Software Defined Hybrid Wireless Internet of Things (SDHW-IoT) to help

relieve network congestion. The technique entailed developing and resolving an optimization problem based on Mixed-Integer Linear Programming (MILP) using Gurobi, as well as simulating Opt-ACM in Mininet-Wifi to compare its performance to both conventional (OLSR and OSPF) and cutting-edge (FACOR and EASDN) methods. Opt-ACM outperformed other approaches in terms of Packet Delivery Ratio (PDR) and Packet Loss Ratio (PLR) efficiency by 9.47 % and 12.32 %, respectively, as well as an average improvement of 26.77 % in Average Delay and 33.10 % in Average Jitter. Assumptions made during optimization modeling and probable hardware/software limits in emulation may be regarded as the limitations of this study.

The paper [19] used the Naked Mole-Rat (NMR) Algorithm to deploy controllers in large-scale SDN networks optimized for latency, load balancing, and other factors. While ensuring load balancing and minimizing Switch-Controller and Controller-Controller latency, the NMR algorithm, a population-based *meta*-heuristic technique, was used to identify the best controller locations. When used with the Ernet and Savvis standard topologies, the NMR method exhibited better controller placement outcomes than the Bat method. Further research was required to determine the scalability and applicability of NMR algorithm in more complicated and varied network topologies.

The article [20] created a dynamic switch-controller mapping model for SDN, which supported load balancing and improved switch-controller stability. The authors described the issue as a convex quadratic programming model, considering both load balancing and assignment stability, and creating a heuristic approach for large-scale networks. Compared with recent academic work, the load balancing was only slightly improved, but the switch-controller assignment stability was much improved (by about 91 %). One of the drawbacks of the suggested model could be implementation complexity or potential restrictions in particular network settings.

The Hybrid Algorithm tried to increase throughput for TCP packet delivery in the Tree and Dumbbell network topologies [21]. The system combined static and dynamic load balancing strategies and adjusted according to the unique properties of the Tree and Dumbbell topologies, resulting in better packet distribution and resource use. The Hybrid Algorithm had remarkable performance in TCP packet transmission, highlighting its well-designed method for increased throughput in the analyzed network topologies. Further research was necessary to determine whether the Hybrid Algorithm can be used for topologies other than Tree and Dumbbell and how well it performs in various network scenarios.

A multi-controller architecture was proposed in the study [22] to improve network performance and test load balancing methods in SDNs. Response time, transaction rate, and throughput performance analysis of the least load balancing, round robin, weighted round robin, and random algorithms were performed. Compared with the single controller topology using the unexpected load balancing algorithm, the multi-controller topology reduced the reaction time by 30.12 %, improved the transaction processing rate by 39.44 %, and improved the throughput by 10.56 %. The scope of the study may include some feasible load-balancing situations and network configurations in sophisticated SDN settings.

A hierarchical multi-controller deployment algorithm that considered load balancing, request delay, and control plane dependability was designed. The study [23] increased the scalability of SD-WAN. The work developed a modeling strategy for SD-WAN and used an upgraded Louvain algorithm to identify hierarchical community structure. Compared with conventional methods, the suggested method improved load balancing, lowered request delay, and increased control plane reliability in SD-WAN. The shortcomings may include particular network scenarios that need to be considered and potential performance changes based on network size and characteristics.

Cloud services have doubled the computing power of mobile devices, enabling operators to execute complex programs. However, the random change of system topology also makes the load balancing problem of network congestion very complex. To achieve secure IoT communication between smart devices, a routing protocol was proposed in [24] that combined multiple alternative paths for adaptive control of network congestion and efficient load balancing in mobile ad hoc networks. The simulation results showed that the proposed method effectively extended the network lifespan by combining load balancing, congestion control, and energy-saving algorithms.

The distributed SDN architecture allows multiple controllers to manage different parts of the network, helping to ensure the efficient and stable operation of the modern SDN architecture. To solve the high switch migration costs and load imbalance in SDN multi-controllers, the research [25] proposed a controller load prediction switch migration strategy based on deep learning and a load balancing algorithm based on migration switch selection and migration decision algorithms. Compared with the distributed decision migration strategy, this method reduced the load variance from 0.02 to 0.004.

Cloud computing has given birth to cloud data centers, which constitute virtual resources. The main problem of cloud data center is to handle millions of concurrent requests/loads from users. To achieve load balancing between virtual machines, the research [26] proposed an improved central load balancer algorithm to avoid virtual machine overload and under-load. The simulation results showed that the proposed method outperformed the comparison methods in terms of data center processing time, response time, and total cost.

The main differences between the above study and the proposed research method are shown in Table 1. Compared with other studies, the GBSO method proposed in this study has advantages in dynamic response and resource utilization efficiency, which helps to improve resource consumption and network efficiency. The experimental results show that the GBSO algorithm reduces the load index to 0.6 after load balancing, indicating its ability to quickly adapt to traffic changes and balance load distribution. In addition, the GBSO algorithm has shown good performance in accuracy, precision, recall, and F1 score indicators, verifying the efficiency of its decision-making process.

2.1. Research on multi-controller

Multi-controller load balancing technology in computer networks has attracted much attention because of its potential in scalability, performance and fault tolerance in large-scale distributed systems. To optimally divide network traffic or computing jobs among numerous resources, this method entails the coordination of several controllers. For instance, in data centers, researchers has looked into algorithms that dynamically distribute incoming requests to controllers depending on workload, resource utilization, and response time [27]. It can use resources more effective and minimize overload on any controller. Similarly, multi-controller load balancing in SDN has been investigated to enhance network management and boost overall system responsiveness. In addition, the research focuses on adaptive load-balancing algorithms, which can adapt to the changing business model and ensure the optimal task allocation in the changing environment [28]. These studies help created a reliable and effective load-balancing solutions, especially in complex networks with high traffic volumes and dynamic resource consumption.

2.2. Load balancing problem

Load balancing is an important problem in computer science and network engineering. It involves effectively allocating tasks, computing load or network traffic among various resources. While maximizing resource consumption and system performance, the goal has to avoid overusing any single resource. Unevenly distributed workloads can cause performance bottlenecks and inefficiencies in various contexts, including data centers, the cloud, Content Delivery Networks (CDNs), and distributed systems [29]. Load balancing algorithms ensure that all resources are used as efficiently as possible while minimizing response

Table 1
Summary table of research methods.

Literature	Research method	Advantage	Limitation
[11]	DRL	Real time routing optimization without prior information	Network complexity affects performance
[12]	Blockchain, QUARK algorithm CapsNet	Combining multiple technologies to enhance performance and security	Complex implementation, relying on multiple technologies for collaboration
[13]	SAPKM	Reduce average load	Limited scalability
[14]	GAIMO	Optimize resource utilization and reduce communication overhead	Sensitive to initial parameters and complex large-scale networks
[15]	Latency-based load balancing method	Reduced latency and improved network performance	Unclear implementation details
[16]	IACO	Improved throughput and fast response time	The dynamic change network needs to be strengthened
[17]	DHAP	Improve controller performance and network efficiency	May overlook other network optimization factors
[18]	Opt-ACM	Improve PDR, PLR, Delay and jitter efficiency	Modeling assumptions and simulation limitations
[19]	NMR	Identify the optimal controller position	Further research is needed on the scalability and applicability of performance
[20]	Dynamic switch controller mapping model	Improve allocation stability	Implement complexity or specific network settings restrictions
[21]	Static and dynamic load balancing strategies, hybrid algorithms	Improve TCP packet transmission throughput	Further research is needed to determine whether it is applicable to other topologies
[22]	Multi-controller architecture	Improve network performance	Limited load balancing and network configuration
[23]	Hierarchical multi-controller deployment algorithm	Improve load balancing and reliability	Specific network scenarios and performance changes
[24]	Adaptive routing protocol	Extend network lifespan	The overall algorithm has a high complexity
[25]	Deep learning	Significant reduction in load variance	Generalization ability may be poor
[26]	Central load balancer algorithm	Optimization of processing time, response time, and total cost	Slow response speed to dynamic changes in load
Research method	GBSO	Improve resource utilization efficiency, accelerate response speed, and optimize load distribution between network segments	High computational complexity

time and preventing service degradation or downtime. Some available load balancing techniques include round-robin, weighted round-robin, least connections, and least response time. Each has advantages and disadvantages. The main goal of this research field is develop complex algorithms that can adjust to dynamic shifts in workload and resource availability.

Furthermore, network topology, resource capacity, and communication costs must be taken into account by load-balancing algorithms. The development of SDN and virtualization has increased the demand for complex load balancing systems, which can effectively control the distribution of virtual resources in changing environments [30]. In various computer and networking applications, solving load balancing is crucial for effectively using resources, improving system response ability and improving user experience.

2.3. Load balancing algorithm

To solve load distribution in complex systems, the load balancing algorithm GBSO combines the concepts of Genetic Algorithms (GA) and Bird Swarm Optimization (BSO). This method uses genetic operators such as selection, crossover and mutation to form a number of potential load balancing solutions through evolution. The system also guides ideal load distribution patterns by taking cues from the flocking behaviors of birds. Birds represent potential answers. The location of birds reflects the burden distribution among various resources [31]. The method imitates how birds communicate, allowing solutions to converge toward load-balanced states while preventing resource saturation. Birds move into new positions during each iteration based on fitness assessments of load distribution solutions and genetic operations that produce fresh solutions. By combining the ideas of genetics and BSO, the method can better identify close-to-ideal load-balancing configurations in dynamic contexts with various workloads and resource availability. To produce effective load distribution techniques that maximize resource utilization, reduce response time, and generally enhance system performance, GBSO aims to balance exploring and utilizing potential solutions. Adding genetic operators and behaviors derived from birds provides load-balancing algorithms with adaptive capabilities, which makes them well-suited for instances where conventional approaches might struggle to handle the complexity of contemporary networked systems [32].

3. Material and methods

This research presents the GBSO, a multi-controller-based load-balancing method. After analysis and problem modeling, switch selection and target controller selection segments are used for selection.

3.1. Problem modeling

A large number of different IoT devices cannot be managed and unified through a single controller. Therefore, the resource limitation of a single controller is solved by multi-controller design. In recent research, it is a key to improve the effectiveness of controller clusters by load balancing between each controller. The schematic illustration of multi-domain collaborative traffic network is shown in Fig. 1.

Through dynamic switching, the connection between switch and controller in this study is remapped. The controller in the three identities of the multi-controller SDN is equal, enslaved person, and enslaver. The switch flow table can only be updated by the master controller among the domain's controllers. In other words, there may be one or more equal slave controllers, but there can never be more than one master controller for a switch. An analogous or enslaved person controller can take over, when the master controller malfunctions. Switch migration refers to the process of moving the main controller of the switch.

The three zones, C1, C2, and C3, comprise the controller layer. Each gadget connects to a switch. Therefore, three zones of 7 controller-managed buttons represent all the devices. Switch S5 and controller C1 are connected, as shown in Fig. 2. S5 slave controller is the remaining controller, while C1 is the master controller of S5. When the load of central controller C1 unexpectedly rises above the predetermined load threshold at a specific time, switch migration is necessary. Additional slave controllers are picked for migration, with the lighter-loaded C3 being used. This study dynamically examines the switch and controller of the GBSO identification of the migrated button.

In the SDN region, each controller node is set to $c = \{c_1, c_2, c_3 \cdots\cdots, c_N\}$, and every switch is $C = \{S_1, S_2, S_3 \cdots\cdots, S_N\}$. The revised algorithm technique is employed for migration to accomplish load balancing when a controller is overloaded.

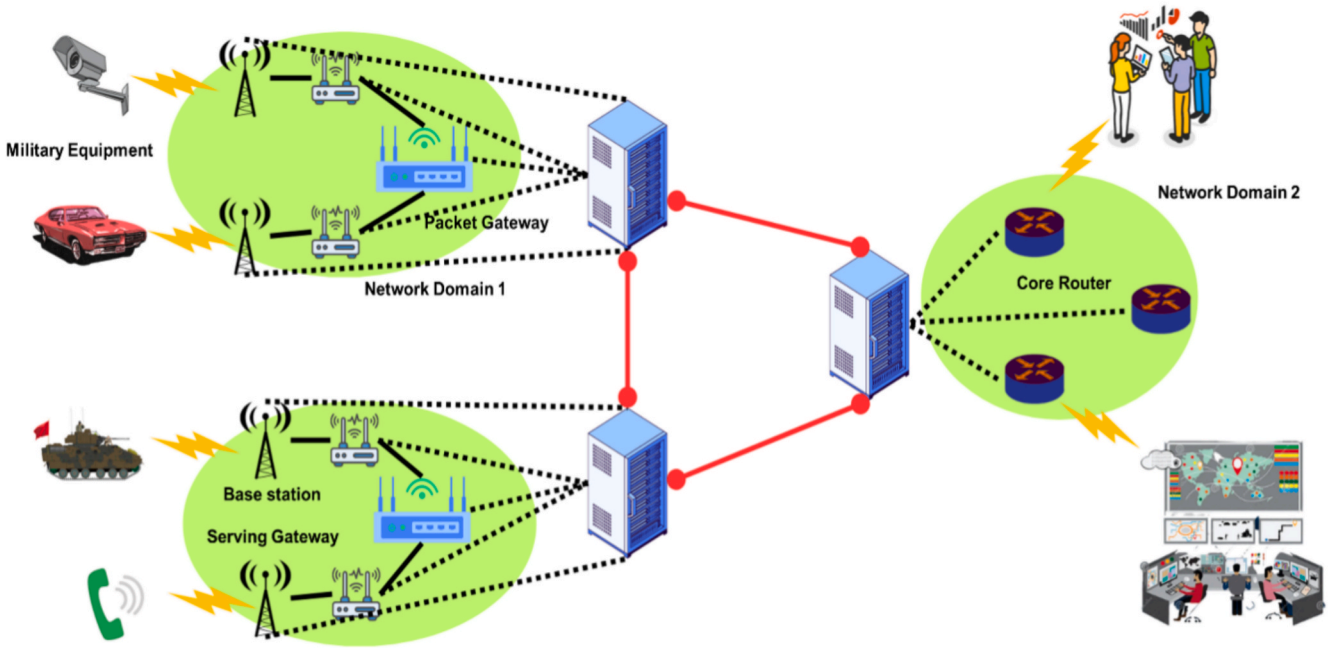


Fig. 1. Multi-domain collaborative traffic network schematic diagram. .png.
Source. https.//media.springernature.com/lw685/springer-static/image/chp%3A10.1007%2F978-3-030-69893-5_9/MediaObjects/500156_1_En_9_Fig. 2_HTML

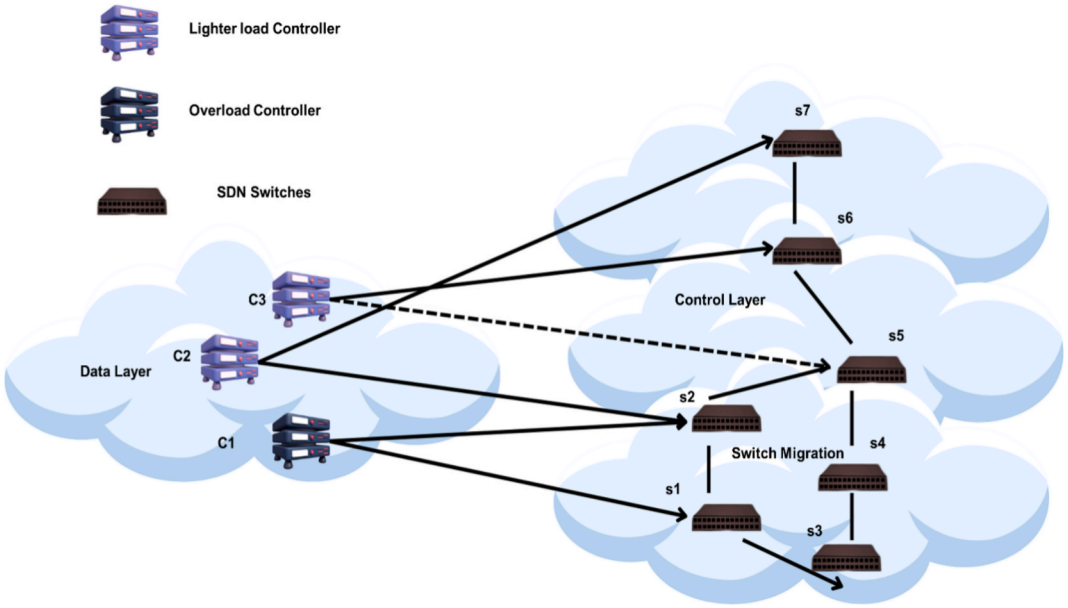


Fig. 2. A multi-controller SDN diagram .
Source. https.//www.mdpi.com/symmetry/symmetry-13-01901/article_deploy/html/images/symmetry-13-01901-g004.png

Example 1. Switch distribution matrix is $L = (l_{ij})_{M^*N}$, which depicts how the controller and switch are mapped out $l_{ij} = 1$, implied C_i, *and* controlled S_j. $l_{ij} = 0$ signifies that the *button* R_j is not controlled by a controller C_i.

Example 2. *The* C_i *of the controller is in-packet load* $\forall_i$. *Equation* (1) *represents the total number of managed packets per switch in the message rate.*

$$\forall_i = \sum j \in S_{C_j} \forall \tag{1}$$

Example 3. *Index of packet-in load* O_i *of the controller* C_i, *equation* (2) *measures how many packets are being received. It is compared with to its processing ability* $\forall_{max}$.

$$O_i = \frac{\forall_j}{\forall_{max}} \tag{2}$$

Example 4. *Index of the typical packet-in load* O_{avg} *in all controller clusters i, as shown in equation* (3).

$$O_{avg} = \frac{1}{|n|} \bullet \sum_{i \in c} O_i \tag{3}$$

Example 5. *For each controller within the cluster, the packet-in is specified. The level of load balancing equation* (4) *is as follows. The packet-in of the controller cluster is in the load standard deviation*:

$$\tau = \sqrt{\frac{1}{|n|}\sum_{i\epsilon c}(O_i - O_{avg})^2} \tag{4}$$

As a result, the best-possible objective function is represented by:

$$S_{best} = GBSO_{best} \tag{5}$$

$$O_i < 0.9\lambda_i \tag{6}$$

$$l_{ij}\epsilon\{0,1\}\lambda_{i,j} \tag{7}$$

$$\sum_{i\epsilon G} l_{ij} = 1\lambda_j \tag{8}$$

The primary purpose of the model optimization is given in Equation (5). S_{best} is the best route for relocating the overload controller identified by the GBSO. Equations (6) to (8) show the primary purpose of the model constraint functions. Equation (6) is also a load-balancing trigger, because it ensures that the load index of all controllers is less than 0.9 to prevent controller failure. The connection status of each network device is represented by Equation (7). Equation (8) ensures the Open Flow transition throughout the cluster. G possesses a single master controller.

3.2. Genetic-bird swarm optimization

GA:

A group of evolutionary algorithms called GAs are motivated by the natural selection. They are utilized to address optimization and search issues. In GA, a population of potential solutions (typically depicted as chromosomes or individuals) goes through selection, crossover (recombination), mutation, and evaluation to evolve and improve over generations.

Initialization: The starting population that generates potential answers (individuals or chromosomes). Usually, these answers are just prospective solutions represented as binary digit strings or different data structures.

Evaluation: The fitness function is used to evaluate the effectiveness of each potential solution in solving the current problem. The fitness function directs the algorithm to favor superior options in the population.

Selection: Based on the fitness level, individuals are chosen from the general population to create a mating pool. The survival of the fittest is simulated by choosing the solution with better fitness.

Crossover: Individuals selected in the mating pool mix their genetic material to create new children. The crossover process, which adds variation to the population, is comparable to biological reproduction.

Mutation: Some members of the offspring population experience arbitrary alterations to their genetic makeup (mutations). The algorithm avoids the local optimal solution by mutation and introduces more exploration.

Replacement: The added population for the following iteration includes the offspring population and some surviving members of the previous generation.

Termination: The algorithm continues to cycle the above steps until one or more termination conditions are satisfied (such as arriving at an acceptable result or reaching a certain number of generations). Fig. 3 depicts the GA.

Bird Swarm Algorithm (BSA):

The *meta*-heuristic algorithm, known as "Bird Swarm Optimization", is motivated by the flocking behavior of birds. It involves a computer-generated group of "birds", which represent the possible answers to a specific optimization problem. These birds use iterative communication and positioning adjustments to find the best answer. They are inspired by their own unique experiences and the collective wisdom of their neighbors. In the exploration and development phase of the algorithm, birds find new parts of the solution space while mining promising areas. BSO dynamically balances the trade-off between exploration and exploitation by combining attraction to the best solution so far with repulsion from other birds. As a result, it converges to optimal or nearly optimal solutions for challenging optimization problems. Fig. 4 depicts the structure of BSO. BSO is a powerful optimization technique with attributes including an easy process, good expansibility, etc. N means birds may fly and search for food. $Z_x^h(x \in [1,2,\cdots..N])$. The $x-th$ bird at h is described. The following are some examples of bird behavior, as shown in equation (9):

$$Z_{x,y}^{h+1} = Z_{x,y}^{h} + (P_{x,y} - Z_{x,y}^{h}) * D * rand(0,1) + (H_{x,y} - Z_{x,y}^{h}) * S * rand(0,1) \tag{9}$$

The following are examples of vigilance behavior, as shown in equation (10):

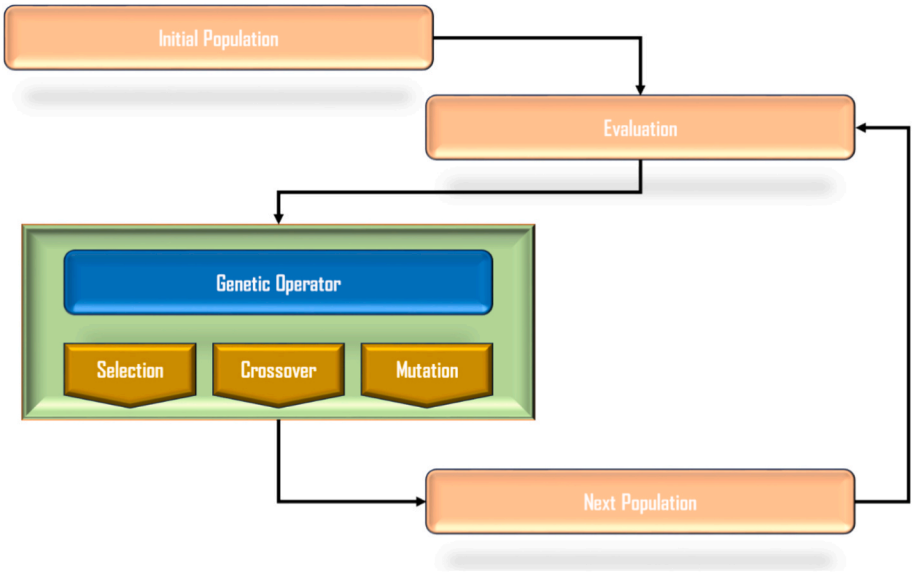


Fig. 3. Structure of genetic algorithm.

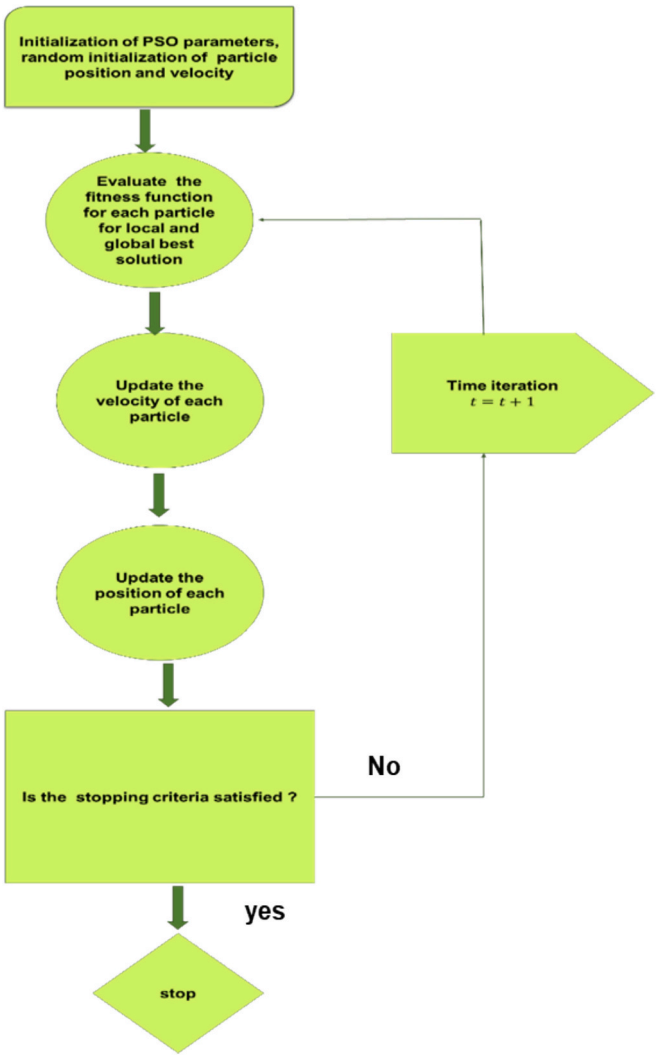


Fig. 4. Structure of BSO.

$Z_{x,y}^{h+1} = Z_{x,y}^{h} + B\,(mean_y - Z_{x,y}^{h})\,*rand(0,1) + B_2\,(P_{x,y} - Z_{x,y}^{h})\,*\,rand\,(-1, 1)$ (10).

B1 and B2 have the following mathematical definitions, as shown in equations (11) and (12):

$$B1 = b1*exp\left[-\frac{p_{Fit_x}}{sum\,Fit + \in}*N\right] \tag{11}$$

$$B2 = b2*exp\left[\left[\frac{p_{Fit_x} - p_{Fit_k}}{|p_{Fit_k} - p_{Fit_x}| + \in}\right]*\frac{N*P_{Fit_k}}{sum\,Fit + \in}\right] \tag{12}$$

The following sentence describes flight behavior, as shown in equations (13) and (14):

$$Z_{x,y}^{h+1} = Z_{x,y}^{h} + randn(0,1)*Z_{x,y}^{h} \tag{13}$$

$$Z_{x,y}^{h+1} = Z_{x,y}^{h} + \left(Z_{k,y}^{h} - Z_{x,y}^{h}\right)*FL*randn(0,1) \tag{14}$$

FL is in [0, 2].

The hybrid optimization approach GBSO combines BSO and GA methods. It is frequently employed to resolve challenging optimization issues. The GBSO algorithm combines the evolutionary features of GA with the social behavior of BSO. Each solution is represented as a "bird" in a swarm. The GBSO algorithm is generally outlined in Algorithm 1.

The following are the steps of the algorithm. Initialization: Create a flock of birds (solutions) in the search space by initializing with random values. Fitting evaluation: Evaluate the fitness of each bird in the population according to the objectives to be optimized. Particle motion: Update equation (15) using BSO to update the speed and position of each bird, taking into account their previous position, speed, personal best results and global best results. Reproduction: Based on genetic operators (crossover and mutation), new progeny are produced by choosing birds for reproduction, depending on their fitness scores. The next generation replaces the aging population. Termination: Perform steps 2 to 5 until the convergence conditions are met or the predetermined number of repetitions is reached.

The basic BSO equation is modified to provide the velocity update equation in GBSO:

$$V_{ij}(t+1) = w{\cdot}V_{ij}(t) + c_1{\cdot}r_1{\cdot}(Pbest_{ij} - X_{ij}(t)) + c_2{\cdot}r_2{\cdot}(Gbest_{ij} - X_{ij}(t)) \tag{15}$$

$V_{ij}(t+1)$ is the speed i of the bird in dimension j at iteration $t+1$. The weight of inertia is w. $c_1 and c_2$ are cognitive and social learning coefficients, respectively. $r_1 and r_2$ are random values. $Pbest_{ij}$ is the i individualized optimal posture of the bird in dimension j. $Gbest_{ij}$ is the top spot on a global position in dimension j.

Algorithm 1: Genetic-bird swarm optimization (GBSO)

Step 1: Initialize:
Define the population size (pop_{size})
Define the number of generations ($max_{generations}$)
Initialize a random population of bird positions and velocities
Set the mutation rate ($mutation_{rate}$)
Step 2: For generation in 1 to $max_{generations}$:
Evaluate the fitness of each bird:
For each bird in the population:
Calculate the fitness of the bird based on the problem
Select parents for reproduction:
Perform tournament selection or roulette wheel selection to choose parents
Create new offspring:
Step 3: For i in range (pop_{size}):
Randomly select two parents from the selected parents
Perform crossover between parents to create a new child
Apply mutation to the child with probability $mutation_{rate}$
Add the child to the new population
Replace the old folk with the new populationEnd loopSelect the best solution from the final population based on fitnessOutput the best solution

3.3. Switch selection

The controller load fluctuation is described. Because the switch selection algorithm only selects a switch according to part of the load, without considering the network load state, this problem arises. The conditions under which overloading requires multiple load balancing operations are also described. The situation is caused by the switch selection algorithm, which only considers the load on the switches while ignoring the load on an overloaded controller. Thus, when selecting buttons, the load of a jammed controller and control device will reduce the balance time to one time. The switches selection process is displayed as algorithm 2. The overloaded controller is taken as input.

A decision or action is selected using a switch selection algorithm depending on the value of a variable or expression. Programming languages frequently use it to implement conditional branching. The fundamental concept is to determine the value of an expression before executing the relevant code block. A switch statement compares an input value to several cases. Each scenario illustrates a potential input value. If the input matches the case, the relevant code snippet of the case is executed. If there is no case match, the default code block may be run.

Algorithm 2: switch selection

Switch (expression):
Case $value_1$:
Code to execute when expression equals $value_1$
Break;
Case $value_2$:

(continued on next page)

(*continued*)

Algorithm 2: switch selection

Ode to execute when expression equals $value_1$
Break;
More cases…
Case $value_1$:
Code to execute when expression equals $value_1$
Break;Default:Code to execute when no cases match the expressionBreak;

3.4. Target controller selection

Target controller selection decides which control or decision-making algorithm is best for a system based on predetermined standards or conditions. This is common in engineering control systems, robotics, and automation fields, where it may be necessary to use numerous controllers to manage various operating situations or jobs efficiently. According to specific requirements or conditions, the process of selecting the best control algorithm or strategy among different controllers to adjust the behavior or performance of the system is called target controller selection. This technique provides load balancing by dividing GBSO among multi-controllers. The algorithm chooses a target controller for each task based on predefined criteria, such as controller workload, latency, or available resources. It is assumed that workload, latency, and resource availability constitute metrics, and the target controller is selected based on the metric with the lowest value.

$$metric = w_1 * normalized_workload + w_2 * normalized_latency + w_3 * normalized_available_resources \quad (15)$$

For *normalized_workload*, the workload of the controller has been adjusted between 0 and 1. *normalized_latency* is the latency of the controller normalized to fall within 0–1. *normalized_available_resources* is the resource availability of the controller normalized to a value between 0 and 1. $w_1, w_2, and w_3$ are the weights attributed to workload, latency, and available resources, respectively. The total should be 1. The following is how to use algorithm 3 to implement this equation for the target controller selection.

Algorithm 3: Target controller selection

Select target controller(controllers)
Lowest metric = infinity
Target controller = null
For each controller in controllers:
Normalized workload = normalize (controller. Workload)
Normalized latency = normalize (controller. Latency)
Normalized available resources = normalize (controller. Available resources)
Metric = w_1*normalized_workload + w_2*normalized_latency + w_3*normalized_available_resourcesIf metric < lowest metric:
Lowest metric = metric
Target controller = controller
Return target controllerNormalize
(value)
Normalize a weight to a range between 0 and 1.
For simplicity, assume value ranges are known for each attribute.Normalized value =
(value − min value) / (max value − min value)
Return normalized value

The select target controller function in this pseudo-code uses the above equation to calculate the metric for each controller, and then selects the controller with the lowest metric as the target controller of GBSO.

4. Performance evaluation

In the experimental setup, the computer runs Ubuntu 18.04.3 and has an i7-6500 processor and 12 GB of RAM. It uses OVS (open virtual switch) for switches, Floodlight as the controller, and Mininet 2.2.1 as the simulator. OpenFlow V1.3 is used for communication, with UDP traffic produced by D-ITG. The UDP load type has a 1024-byte packet size. The proposed method is evaluated when a controller carries a hefty weight. Three metrics: packet-in response time, throughput, and controller load rate, are used to compare the controller performance before and after migration.

The study evaluates the impact of migrating to a multi-controller-based load-balancing algorithm in computer networks. Packet-in response time, throughput, and controller load rate are the performance measures considered. The proposed method GBSO is compared with the other two optimization algorithms, namely Ant Colony Optimization (ACO) and a hybrid approach combining GA with ACO (GA-ACO). The GBSO algorithm optimizes the load distribution among multiple controllers, enhancing network efficiency. By employing genetic and swarm intelligence principles, it seeks to achieve superior performance compared with the ACO and GA-ACO methods. The evaluation includes the pre-migration and post-migration scenarios, providing insights into the effectiveness of the algorithm in enhancing network parameters and maintaining low packet response time, while effectively balancing the controller load.

Fig. 5 depicts the throughput comparison. GBSO seeks to improve the throughput by iterating the improved solution, effectively converges through genetic evolution and bird swarm behavior, and improves the quality of the solution (when applied to the problem that generates 3,800 values).

Fig. 6 depicts the load index comparison between before and after load balancing. Before load balancing, the load index was greater, suggesting that the jobs were distributed unevenly among the system resources. The load index dropped to GBSO-0.6 after load balancing, indicating that the load distribution was balanced. When each resource handled a proportional share of work, a load index value of 0.6 indicated that the system operated in a balanced and efficient manner. The risk of resource overuse or under-use was reduced due to this enhancement, which also guaranteed optimal performance.

Figs. 7 and 8 depict the results of the experimental simulation. Table 2 depicts the timing of packet-in responses before and after the controller migration. The latency between the network event and the processing of the packet by the controller refers to the packet-in response time a SDN, such as GBSO. Due to greater controller proximity, improved network performance, and responsiveness after migration, this time may be reduced. Due to the optimal distribution of network traffic following load balancing in GBSO with a parameter value of 20, the packet-in response time tended to decrease. When GBSO was set to 18, the response time of a single controller was faster than that of alternative technologies such as ACO and GA-ACO, indicating that GBSO was more effective in controlling controller tasks and network events.

Fig. 9 depicts the moving multiple switches. When using multiple

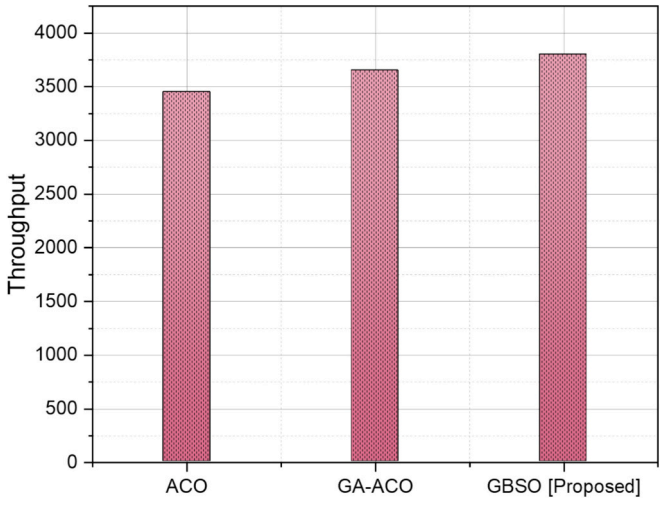


Fig. 5. Comparison of Throughput.

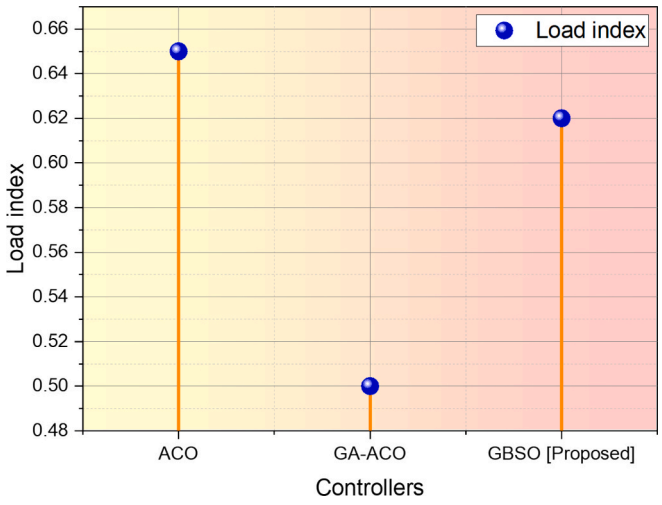


Fig. 6. Load index comparison between before and after load balancing.

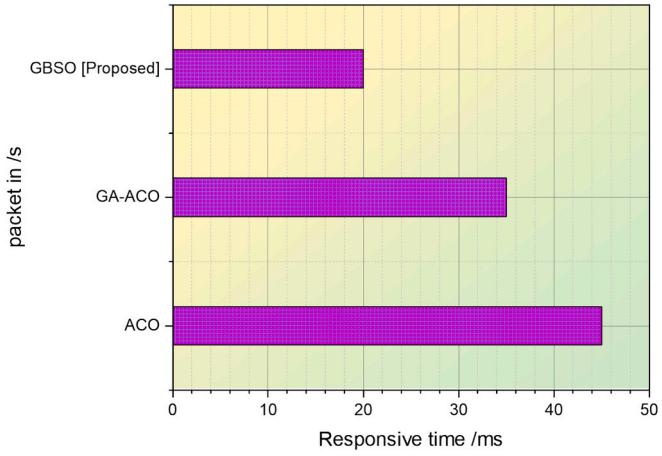


Fig. 7. Comparison of packet-in response time after load balancing.

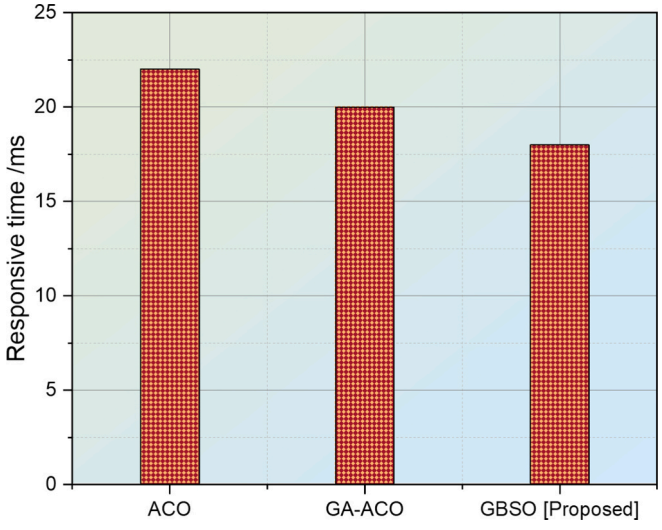


Fig. 8. Comparison of packet-in response time for each controller.

directors, the main technology to solve the load balancing problem is switch migration, although frequent switch migration may reduce the operation of the entire network. The information exchange and

Table 2
Timings of packet-in responses before and after a controller migration.

Packet-in /s	Response time /ms	
ACO	45	22
GA-ACO	35	20
GBSO [Proposed]	20	18

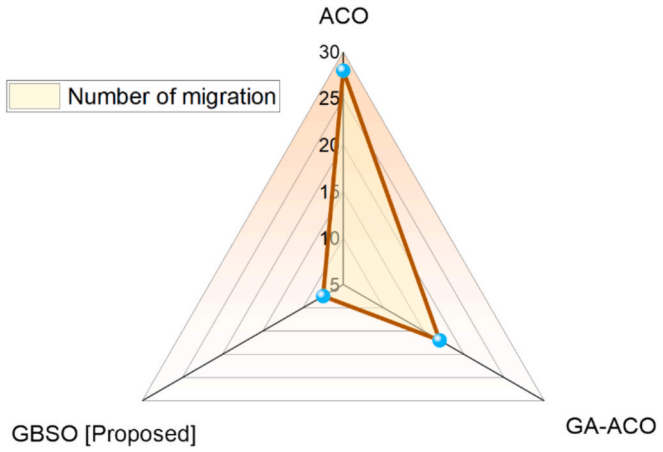


Fig. 9. Number of switches migrated.

migration time of each switch to another controller are practically the same in the current network environment and may be defined as com_m. The central controller controls each switch. The communication overhead can be written as $Com_m{*}Num_m$. Num_m is the total number of migrations required to perform the load balancing. It refers to the number of switch migrations, as communication overhead is load balanced, while com_m is practically the same. After load balancing, GBSO has the fewest migrations and communication overhead.

The performance of this method is evaluated in terms of accuracy, precision, recall, F1 score, load balance rate, and network latency. Naive Bayes [33], Support Vector Machine (SVM) [33], Krill Herd Algorithm (KHA) [34], and Modified Affinity Propagation Algorithm (PSOAP) [34] are compared with the proposed method.

Load balance rate measures the effectiveness of load balancing algorithm in ensuring the fair distribution of network traffic among multiple controllers. A higher load balance rate indicates that the algorithm successfully reduces traffic differences between controllers, improving network performance. Fig. 10 depicts the load balance rate. Table 3 describes the specificity value. The suggested method performed better than the existing KHA-0.35 and PSOAP-0.3. It demonstrates that the multi-controller-based load balancing algorithm GBSO-0.2 approach in computer networks is successful.

$$LoadBalanceRate = \frac{ActualLoadBalanceAchieved}{MaximumPotentialLoadBalance} \tag{16}$$

Data packets traversing a computer network undergo latency, that is, the temporal latency. The key is to consider the impact of algorithm selection on network latency. Good traffic is an essential distribution algorithm that should reduce the latency for applications and end users. Fig. 11 and Table 4 describe the network latency values. The proposed method performed better than the existing KHA-10 % and PSOAP-9 %. The results show that GBSO-8 % technology is effective for multi-controller-based load balancing in computer networks.

The results measure the accuracy of load balancing algorithm prediction or decision. It evaluates how effectively the algorithm distributes incoming network traffic among several controllers to meet load-balancing performance objectives. Fig. 12 and Table 5 depict the accuracy value. The proposed method is superior compared with existing

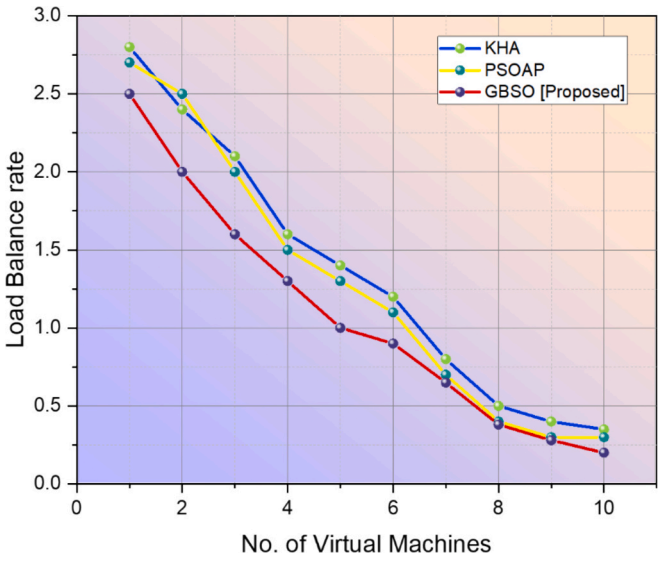


Fig. 10. Outcome of load balance rate.

Table 3
Values of load balance rate.

Number of virtual machines	Load balance rate		
	KHA [34]	PSOAP [34]	GBSO [Proposed]
1	2.8	2.7	2.5
2	2.4	2.5	2
3	2.1	2	1.6
4	1.6	1.5	1.3
5	1.4	1.3	1
6	1.2	1.1	0.9
7	0.8	0.7	0.65
8	0.5	0.4	0.38
9	0.4	0.3	0.28
10	0.35	0.3	0.2

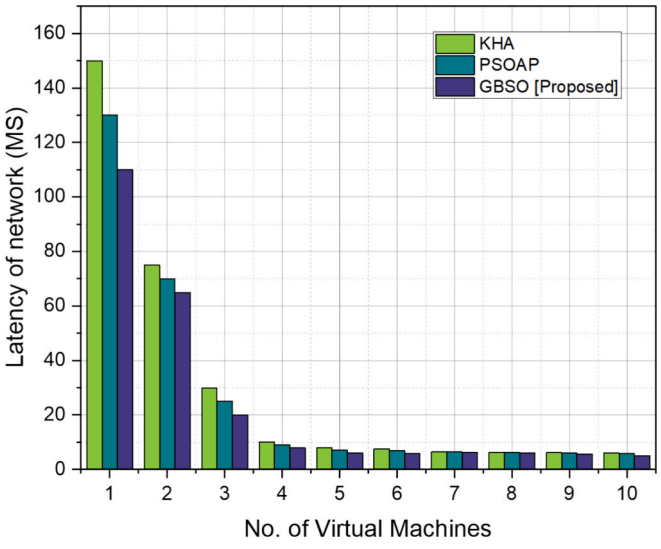


Fig.11. Outcome of latency.

Naive Bayes-76.56 % and SVM-69.52 %. It shows that the proposed GBSO-92.15 %performed better regarding multi-controller-based load balancing in computer networks.

$$Accuracy = \frac{TotalNumberofInstances}{NumberofCorrectlyClassifiedInstances} \quad (17)$$

Table 4
Network latency.

Number of virtual machines	Latency (ms)		
	KHA [34]	PSOAP [34]	GBSO [Proposed]
1	150	130	110
2	75	70	65
3	30	25	20
4	10	9	8
5	8	7	6
6	7.5	6.9	5.8
7	6.5	6.4	6.2
8	6.3	6.2	6
9	6.2	6.1	5.6
10	6	5.8	5.1

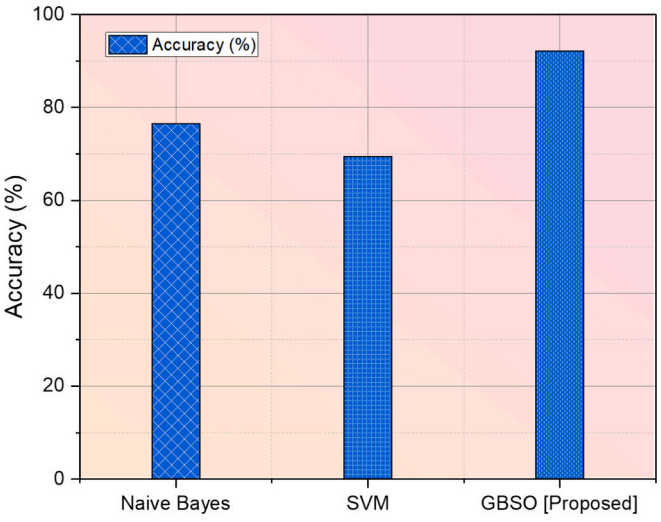


Fig. 12. Outcome of accuracy.

Table 5
Accuracy value.

Methods	Accuracy (%)
Naive Bayes [33]	76.56
SVM [33]	69.52
GBSO [Proposed]	92.15

Precision is a metric, that is, the degree to which an algorithm can distinguish a key instance from other instances it marks as related. Precision in load balancing may be reflected in how well the algorithm prevents the overloading of particular controllers, maintaining a balanced and effective traffic distribution. Fig. 13 depicts the precision result, Table 6 depicts the precision value. The proposal is more significant compared with the existing methods Naive Bayes-82 % and SVM-84 %. It shows that the proposed GBSO-89 % is effective for multi-controller-based load-balancing in computer networks.

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives} \quad (18)$$

Recall, usually refers to as sensitivity or actual positive rate, measures the degree to which an algorithm can distinguish related instances from all actual related instances. Recall in load balancing represents how well the algorithm can maintain optimal network performance by ensuring that no controller is under loaded. Fig. 14 and Table 7 depict the recall value. The proposed method is higher than existing methods Naive Bayes-73 % and SVM-79 %. It shows that the proposed method GBSO-88 % implements better in multi-controller-based load balancing in computer networks.

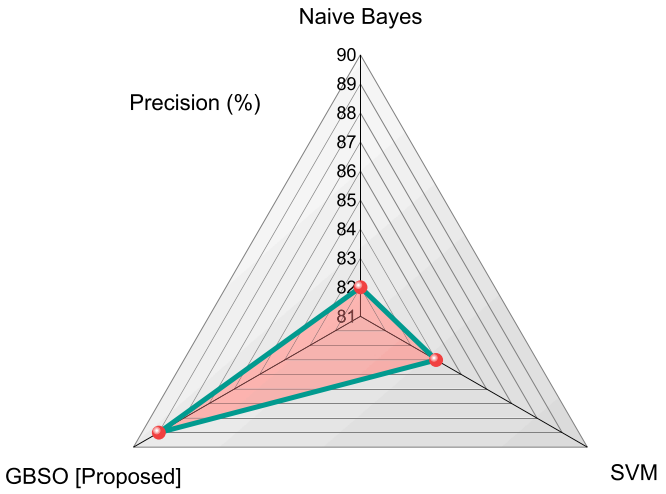


Fig. 13. Outcome of precision

Table 6
Precision value.

Methods	Precision (%)
Naive Bayes [33]	82
SVM [33]	84
GBSO [Proposed]	89

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives} \tag{19}$$

A statistic called the F1 score combines recall and precision into a single number. When precision and recall must be traded off, it is beneficial. F1 score can fairly evaluate the ability of the algorithm, distribute traffic fairly, and prevent overload and insufficient load balance. Fig. 15 and Table 8 describe the F1 score values. The proposed method was better than commonly used Naive Bayes −72 % and SVM-77 %. It demonstrates that the proposed method GBSO-85 % achieves load balancing based on multi-controller in computer networks.

$$F1score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{20}$$

5. Discussion

The "multi-controller-based load balancing in computer networks"

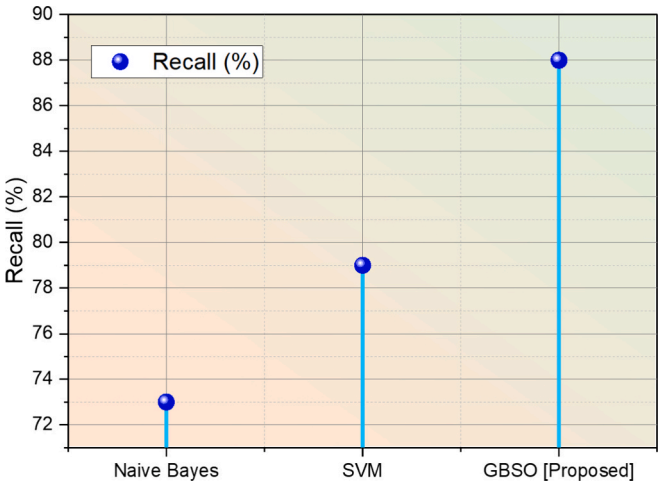


Fig. 14. Outcome of recall Source. author source.

Table 7
Recall value.

Methods	Recall (%)
Naive Bayes [33]	73
SVM [33]	79
GBSO [Proposed]	88

aims to improve network efficiency and resource usage. The accuracy of the Naive Bayes classifier may be constrained by complicated inter dependencies among network parameters when used for load balancing. Due to its computational complexity and potential over-fitting, SVM is helpful to deal with massive network data. The KHA may have convergence problems in dynamic network contexts and offer less-than-ideal solutions. When PSOAP attempts to handle fluctuating network load, similar problems may occur, which may lead to slower convergence speed and increased control overhead. This strategy maximizes the allocation of network traffic over several controllers through the GBSO, allowing for practical resource usage, reducing response time, and improving fault tolerance. Due to the adaptability and dynamic response ability of GBSO, the load balancing algorithm can quickly and effectively adjust the strategy. As a result, network performance, congestion, and scalability are all improved. The combination of multi-controller load balancing with GBSO optimizes resource utilization. It allows the network to gracefully absorb traffic variations, making its architecture more robust and effective. However, although the GBSO algorithm combines the advantages of GA and PSO algorithm, it also inherits the computational complexity of both. In large-scale network environments, the computational cost of GBSO algorithm may significantly increase, especially when dealing with high-dimensional network state and traffic data. Therefore, in future research, the algorithm needs to be further optimized and accelerated. For example, GBSO is combined with other optimization algorithms such as differential evolution algorithm and simulated annealing algorithm to form a hybrid optimization strategy to improve the global search ability and convergence speed of the algorithm. Alternatively, by utilizing multi-core processors or distributed computing architectures, the GBSO algorithm can be

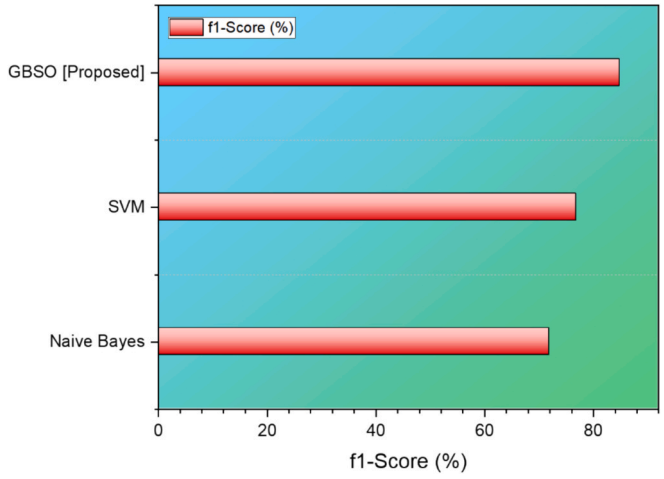


Fig. 15. Outcome of F1 score.

Table 8
F1 score.

Method	F1 score (%)
Naive Bayes [33]	72
SVM [33]	77
GBSO [Proposed]	85

parallelized to reduce computational complexity.

6. Conclusion

Dynamic workloads are complex for traditional load-balancing methods, resulting in inefficient resource utilization and protracted response time. This study emphasized multi-controller-based load balancing while introducing a novel viewpoint using the GBSO method. This method performed well in the selection process including cross segment allocation of switches and target controllers and thorough problem modeling, analysis and evaluation. The proposed method is evaluated when employing the heavy loaded controller. Three metrics were utilized to compare the controller's performance before and after migration: packet-in response time, throughput, and controller load rate. A thorough examination using parameters including accuracy, precision, recall, F1 score, load balance rate, and network latency provides a strong case for the validity of the experimental findings. Integrated with GBSO, the framework achieved optimal detection accuracy of 92.15 %, reduced false positives, and improved adaptability. The report emphasizes the critical part load balancing plays in optimizing computer network performance. In the end, the unique algorithm proposed in this study offers a dependable solution for attaining fair distribution of network traffic, successfully relieving the limitations of existing methods. The network topology and traffic patterns may impact the performance of GBSO algorithm, necessitating possible fine-tuning for various situations. To improve the applicability of GBSO in the dynamic and evolving network environment, the scalability of GBSO to a wider network and its response to real-time changes in network conditions are further explored.

CRediT authorship contribution statement

Fengfeng Guo: Methodology, Investigation, Data curation, Conceptualization. **Ailing Ye:** Validation, Software, Methodology, Conceptualization.

Funding

The key project of Anhui Province's Young Talents Support Program in 2020-Research on Data Transmission Path Selection Load Balancing Based on SDN (project number: gxyqZD2020063); In 2022, the key project of natural science research in colleges and universities of Anhui Provincial Department of Education-research on multi-controller load balancing based on SDN network (project number: 2022AHO52768).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Oke AA. Resilient routing in software defined networks utilizing residue number system and shortest path fast re-routing technique. Kwara State University (Nigeria)); 2021. Doctoral dissertation.
[2] Costin A, Eastman C. Need for interoperability to enable seamless information exchanges in intelligent and sustainable urban systems—Journal of Computing in. Civ Eng 2019;33(3):04019008.
[3] Thantharate A, Paropkari R, Walunj V, Beard C. In: October. Deepslice: A Deep Learning Approach towards Efficient and Reliable Network Slicing in 5G Networks. IEEE; 2019. p. 0762–7.
[4] Giordani M, Polese M, Mezzavilla M, Rangan S, Zorzi M. Toward 6G networks: Use cases and technologies. IEEE Commun Mag 2020;58(3):55–61.
[5] Hamdan M, Hassan E, Abdelaziz A, Elhigazi A, Mohammed B, Khan S, et al. A comprehensive survey of load balancing techniques in software-defined networks. J Netw Comput Appl 2021;174:102856.
[6] Medhi, N. and Saikia, D.K., 2020. Openflow-based multi-controller model for fault-tolerant and reliable control plane. In Smart Computing Paradigms: New Progresses and Challenges: Proceedings of ICACNI 2018, Volume 2 (pp. 61-73). Springer Singapore.
[7] Abdulqadder IH, Zou D, Aziz IT. The DAG blockchain: A secure edge-assisted honeypot for attack detection and multi-controller-based load balancing in SDN 5G. Futur Gener Comput Syst 2023;141:339–54.
[8] Tajbakhsh H, Parizotto R, Neves M, Schaeffer-Filho A, Haque I. In: June. Accelerator-Aware in-Network Load Balancing for Improved Application Performance. IEEE; 2022. p. 1–9.
[9] Tyagi V, Singh S. Network resource management mechanisms in SDN enabled WSNs. A Comprehensive Review—Computer Science Review 2023;49:100569.
[10] Cai CJ, Winter S, Steiner D, Wilcox L, Terry M. " Hello AI": uncovering the onboarding needs of medical practitioners for human-AI collaborative decision-making. Proc ACM Hum Comput Interact 2019;3(CSCW):1–24.
[11] Srivastava V, Pandey RS. Machine intelligence approach: To solve load balancing problems with high quality of service performance for multi-controller based Software Defined Network. Sustainable Comput Inf Syst 2021;30:100511.
[12] Javadpour A, Abadi AMH, Rezaei S, Zomorodian M, Rostami AS. Improving load balancing for data-duplication in big data cloud computing networks. Clust Comput 2022;25(4):2613–31.
[13] Yang K, Guo D, Zhang B, Zhao B. Multi-controller placement for load balancing in SDWAN. IEEE Access 2019;7:167278–89.
[14] Xu A, Sun S, Wang Z, Wang X, Han L. In: July. Multi-Controller Load Balancing Mechanism Based on Improved Genetic Algorithm. IEEE; 2022. p. 1–8.
[15] Babbar H, Rani S, Masud M, Verma S, Anand D, Jhanjhi N. Load balancing algorithm for migrating switches in software-defined vehicular networks. Comput Mater Contin 2021;67(1):1301–16.
[16] Fu Y, Zhu Y, Cao Z, Du Z, Yan G, Du J. Multi-controller load balancing algorithm for test network based on IACO. Symmetry 2021;13(10):1901.
[17] El Kamel A, Youssef H. We are improving switch-to-controller assignment with load balancing in multi-controller software-defined WAN (SD-WAN). J Netw Syst Manag 2020;28:553–75.
[18] Kumar R, Venkanna U, Tiwari V. Opt-ACM: An optimized load balancing-based admission control mechanism for software-defined hybrid wireless-based IoT (SDHW-IoT) network. Comput Netw 2021;188:107888.
[19] Sapkota, A., Dawadi, B.B.R. and Joshi, C.S.R., 2022. Multi-Controller Placement Optimization Using Naked Mole-Rat Algorithm over Software-Defined Networking Environment. Journal of Computer Networks and Communications, 2022.
[20] Al-Tam F, Correia N. Fractional switch migration in multi-controller software-defined networking. Comput Netw 2019;157:1–10.
[21] Avinash S, Srikar AS, Naik VP, Bhaskaran S. In: June. SDN-Based Hybrid Load Balancing Algorithm. IEEE; 2023. p. 1–5.
[22] Mitiku, K., 2022. Improving the Performance of Software-Defined Network Load Balancer Using Open Flow Based Multi-Controller Topology (Doctoral dissertation, ST. MARY'S UNIVERSITY).
[23] Hou X, Muqing W, Bo L, Yifeng L. Multi-controller deployment algorithm in hierarchical architecture for SDWAN. IEEE Access 2019;7:65839–51.
[24] Dalal S, Seth B, Jaglan V, Malik M, Dahiya NS, Rani U, Le DN, Hu YC. RETRACTED ARTICLE: An adaptive traffic routing approach toward load balancing and congestion control in Cloud–MANET ad hoc networks. Soft Comput 2022;26(11):5377–88.
[25] Xiao J, Pan X, Liu J, Wang J, Zhang P, Abualigah L. Load balancing strategy for SDN multi-controller clusters based on load prediction. J Supercomput 2024;80(4):5136–62.
[26] Shetty SM, Shetty S. Analysis of load balancing in cloud data centers. J Ambient Intell Hum Comput 2024;15(1):973–81.
[27] Al-Jaroodi J, Mohamed N, Jiang H. Swanson Middleware infrastructure for parallel and distributed programming models in heterogeneous systems. IEEE Trans Parallel Distrib Syst 2016;14:1100–11.
[28] Yeganeh SH, Kandoo GY. In: a Framework for Efficient and Scalable Offloading of Control Applications. New York, NY, USA: ACM Press; 2012. p. 19–24.
[29] Ye X, Cheng G, Luo X. Maximizing SDN control resource utilization via switch migration. Comput Net 2017;126:69–80.
[30] Yao G, Bi J, Li Y, Guo L. On the capacitated controller placement problem in software defined networks. IEEE Commun Lett 2014;18:1339–42.
[31] Lan W, Li F, Liu X, Qiu Y. A Dynamic Load Balancing Mechanism for Distributed Controllers in Software-Defined Networking 2018;10–11:259–62.
[32] Dorigo M, Birattari M, Stutzle T. Ant colony optimization. IEEE Comput Intell Mag 2006;1:28–39.
[33] Sabarish MM, Mayilvahanan P. A survey of load balancing and security in software defined networks. Annals of the Romanian Soc Cell Biology 2021;25(6):2978–91.
[34] Forghani, M., Soltanaghaei, M. and Zamani Boroujeni, F., Dynamic Optimizing Scheme for Load Balancing in Software-Defined Networks with the Approach of Reducing Energy Consumption. Available at SSRN 4512817.